



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

ASSIGNMENT REPORT

*Submitted in the partial fulfilment for the Course
of*

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

R. Paul David (192572144)

Bhuvaneshwaran J (192511402)

Under the Supervision of

Dr.Uma Priyadarsini P. S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

ASSIGNMENT REPORT

Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

CSA0309 – Data Structures – Slot B

A. Assignment Information

Particular	Details
Department	Computer Science and Engineering
Programme	Computer Science and Engineering
Course Code & Course Name	CSA0309 – Data Structures – Slot B
Academic Year / Batch	2026-2027
Faculty Name	Dr. Uma Priyadarsini P. S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31/08/2026
Date of Submission	02/09/20026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/Red-Black Trees) for ordered, dynamically balanced storage of risk-ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C. CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	Real-time cybersecurity operations, high-volume security-log processing, rapid duplicate-IP detection, risk prioritization, scalable infrastructure, and security monitoring.

B. Assignment Problem / Challenge

This assignment applies Data Structures concepts to a realistic cybersecurity operations problem. The solution must process high-volume security events, identify repeated source IPs quickly, rank suspicious events, remain efficient as the dataset grows, and be evaluated under different load factors and data distributions.

C. Problem Statement

Problem / Case / Design Challenge:

A cybersecurity operations center is developing a real-time threat-detection engine that ingests security event logs (login attempts, IP connections, access requests) at high volume. Each event contains a source IP, timestamp, event type, and risk indicators. The system must process 1,000,000+ events with minimal latency, identify duplicate or repeated IP addresses instantly to detect brute-force or scanning patterns, surface suspicious and high-risk events ahead of routine events, and remain efficient as the number of distinct IPs and events grows. It must also handle varying hash-table load factors and skewed distributions, such as a large share of events originating from a small set of malicious IPs. The proposed C implementation uses a hash table with separate chaining for average $O(1)$ duplicate-IP lookup, an AVL tree for dynamically ordered risk-ranked event storage, and a max-priority queue for continuous highest-risk event retrieval.

D. Requirements and Constraints

- Functional requirements: process high-volume security events; detect repeated source IPs; rank high-risk events; maintain ordered risk data.
- Performance requirements: support experiments with 10,000, 100,000, and 1,000,000 events; evaluate multiple load factors.
- Technical constraints: implementation in C; hash collisions must be handled explicitly; the ordered structure must be self-balancing.
- Reliability: records should not be lost during collisions, and allocated memory should be released correctly.
- Security/privacy: generated IP data is used for testing; a production deployment would require access control, secure log handling, and retention policies.
- Time/resources: benchmark results should be repeatable and recorded from the actual execution environment.
- Sustainability: scalable data structures reduce unnecessary processing and support efficient infrastructure use.

E. Student Work

1. Problem Understanding and Formulation

The problem has two different high-speed requirements: fast source-IP membership/duplicate counting and rapid identification of the most important events. A single structure is not optimal for both, so the design combines hashing, an AVL tree, and a max heap.

Expected outcomes:

- Average $O(1)$ hash-table operations for source-IP duplicate detection.
- $O(\log n)$ AVL insertion with ordered risk traversal.
- $O(\log n)$ priority-queue insertion and $O(1)$ access to the highest-risk event.
- Empirical comparison under multiple load factors and uniform/skewed distributions.
- A modular, compilable C implementation with execution evidence.

Available data consists of generated security events containing source IP, timestamp, event type, risk score, and sequence number.

Assumptions:

- IPv4 addresses are represented as strings of at most 15 characters.
- Higher risk score means higher priority.

- For equal risk, earlier timestamp is given higher priority.
- Benchmark timings depend on the actual machine or online execution environment.

2. Application of Course Knowledge

- Hashing: FNV-1a maps source IP strings to hash-table buckets.
- Collision resolution: separate chaining stores colliding IP records in linked lists.
- Self-balancing tree: AVL rotations maintain logarithmic tree height.
- Priority queue: a binary max heap keeps the highest-risk event at the root.
- Dynamic memory management: calloc, malloc, realloc, and free are used for runtime structures.
- Complexity analysis: expected hash operations are $O(1)$ average case; AVL and heap insertion are $O(\log n)$.

3. Solution / Design / Methodology

The processing pipeline is: Security Event → Hash Table for duplicate-IP tracking → AVL Tree for ordered risk storage → Max Heap for highest-risk retrieval.

3.1 System Flowchart

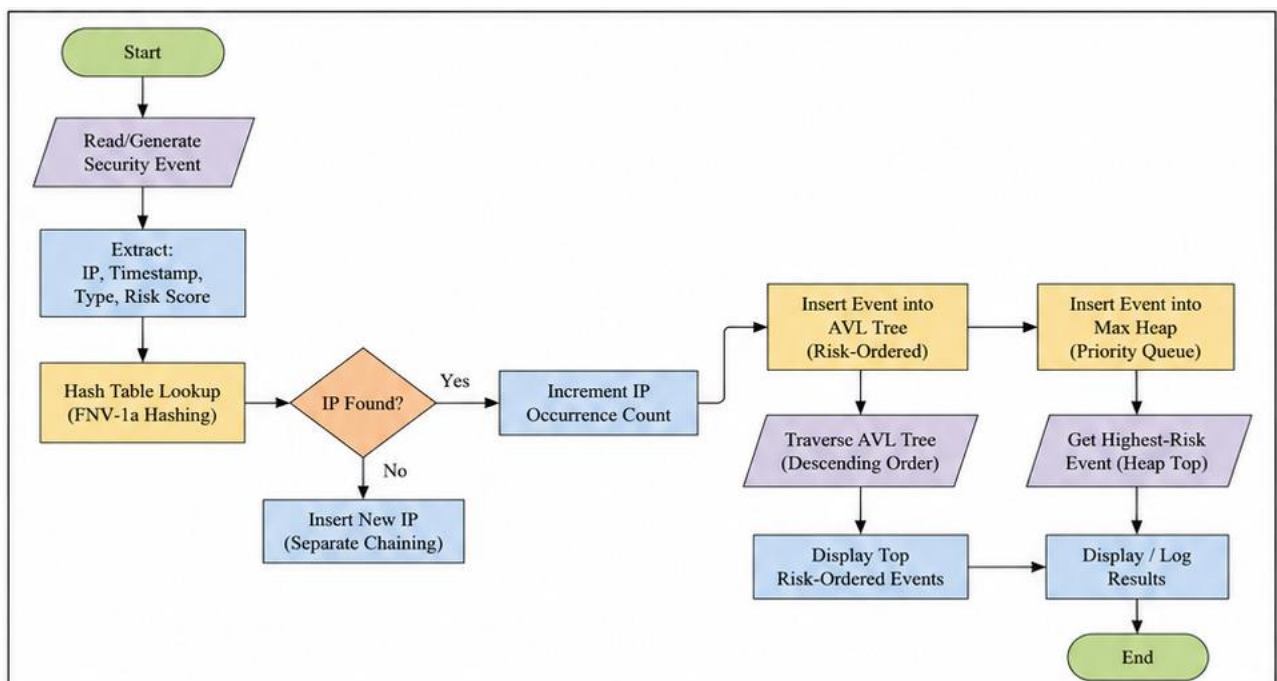


Figure 1: System Flowchart of the Real-Time Cyber Threat Detection Engine.

Data Structure	Purpose	Insertion	Search / Access	Key Feature
Hash Table	Duplicate-IP detection	$O(1)$ average	$O(1)$ average	Separate chaining
AVL Tree	Ordered risk-ranked storage	$O(\log n)$	$O(\log n)$ by tree key	Self-balancing BST
Max Heap	Highest-risk ranking	$O(\log n)$	$O(1)$ peek	Binary max heap

Design rationale:

- Separate chaining provides straightforward collision handling and allows the table to continue operating as chains grow.
- AVL was selected because strict height balancing gives predictable logarithmic ordered operations.
- A max heap was selected because the highest-risk event can be accessed at the root without scanning the complete event set.
- The three structures are maintained together so each performs the operation for which it is most suitable.

4. Use of Modern Tools

The implementation is written in C and is suitable for GCC-compatible compilation. OnlineGDB or another online C environment can be used for functional execution, while a local GCC environment is preferable for the largest benchmark because online sandboxes may impose CPU or memory limits.

- C11-compatible source code.
- GCC-compatible compiler.
- Online C execution environment for functional validation.
- Git/GitHub for source-code submission and version control.
- Benchmark mode producing CSV-style performance measurements.

Evidence of tool usage is provided by the actual execution screenshot included in the Results and Validation section.

5. Results and Validation

Functional test cases:

ID	Test	Input / Configuration	Expected Result	Actual / Status
TC01	Insert unique IP	192.168.1.10	IP stored with count = 1	Pass – actual execution
TC02	Repeat same IP	192.168.1.10 repeated	Duplicate count increases	Pass – actual execution
TC03	Unknown IP lookup	8.8.8.8	Count = 0	Pass – actual execution
TC04	High-risk event	Risk = 99	Appears first in max heap	Pass – actual execution
TC05	AVL risk ordering	Mixed risk values	Highest-risk events appear first	Pass – actual execution
TC06	Hash collisions	Generated IP set	Collisions handled without record loss	Pass – actual execution

Complete C Source Code

The complete 342-line C implementation is reproduced below. It contains the event model, FNV-1a hash table with separate chaining, AVL tree, max heap, integrated engine, synthetic benchmark generator, demonstration mode, and benchmark mode.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <stdint.h>
#define MAX_IP_LEN 16
#define MAX_TYPE_LEN 24
#define DEMO_EVENTS 12
/* ----- Event model ----- */
typedef struct {
    char ip[MAX_IP_LEN];
    unsigned long timestamp;
    char type[MAX_TYPE_LEN];
    int risk;
    unsigned long seq;
} Event;
/* ----- Hash table ----- */
typedef struct HashNode {
    char ip[MAX_IP_LEN];
    unsigned long count;
    struct HashNode *next;
} HashNode;
typedef struct {
    HashNode **buckets;
    size_t size;
    size_t items;
    size_t collisions;
} HashTable;
static uint64_t fnv1a(const char *s) {
    uint64_t h = 1469598103934665603ULL;
    while (*s) {
        h ^= (unsigned char)*s++;
        h *= 1099511628211ULL;
    }
    return h;
}
static HashTable *hash_create(size_t size) {
    HashTable *t = (HashTable *)calloc(1, sizeof(HashTable));
    if (!t) return NULL;
    t->size = size;
    t->buckets = (HashNode **)calloc(size, sizeof(HashNode *));
    if (!t->buckets) { free(t); return NULL; }
```

```

    return t;
}
/* Returns previous count; inserts new IP with count=1 if absent. */
static unsigned long hash_touch(HashTable *t, const char *ip) {
    size_t idx = (size_t)(fnv1a(ip) % t->size);
    HashNode *cur = t->buckets[idx];
    if (cur) t->collisions++;
    while (cur) {
        if (strcmp(cur->ip, ip) == 0) {
            unsigned long prev = cur->count;
            cur->count++;
            return prev;
        }
        cur = cur->next;
    }
    HashNode *node = (HashNode *)calloc(1, sizeof(HashNode));
    if (!node) return 0;
    strncpy(node->ip, ip, MAX_IP_LEN - 1);
    node->count = 1;
    node->next = t->buckets[idx];
    t->buckets[idx] = node;
    t->items++;
    return 0;
}
static unsigned long hash_count(const HashTable *t, const char *ip) {
    size_t idx = (size_t)(fnv1a(ip) % t->size);
    HashNode *cur = t->buckets[idx];
    while (cur) {
        if (strcmp(cur->ip, ip) == 0) return cur->count;
        cur = cur->next;
    }
    return 0;
}
static void hash_free(HashTable *t) {
    if (!t) return;
    for (size_t i = 0; i < t->size; ++i) {
        HashNode *cur = t->buckets[i];
        while (cur) {
            HashNode *next = cur->next;
            free(cur);
            cur = next;
        }
    }
    free(t->buckets);
    free(t);
}

```

```

/* ----- AVL tree ----- */
typedef struct AVLNode {
    Event event;
    int height;
    struct AVLNode *left, *right;
} AVLNode;
static int avl_height(const AVLNode *n) { return n ? n->height : 0; }
static int max_int(int a, int b) { return a > b ? a : b; }
static void avl_update(AVLNode *n) { n->height = 1 + max_int(avl_height(n->left),
avl_height(n->right)); }
static AVLNode *rotate_right(AVLNode *y) {
    AVLNode *x = y->left;
    AVLNode *t = x->right;
    x->right = y; y->left = t;
    avl_update(y); avl_update(x);
    return x;
}
static AVLNode *rotate_left(AVLNode *x) {
    AVLNode *y = x->right;
    AVLNode *t = y->left;
    y->left = x; x->right = t;
    avl_update(x); avl_update(y);
    return y;
}
static int event_cmp(const Event *a, const Event *b) {
    if (a->risk != b->risk) return (a->risk > b->risk) ? 1 : -1;
    if (a->timestamp != b->timestamp) return (a->timestamp < b->timestamp) ? 1 : -1;
    if (a->seq != b->seq) return (a->seq < b->seq) ? 1 : -1;
    return strcmp(a->ip, b->ip);
}
static AVLNode *avl_insert(AVLNode *root, Event e) {
    if (!root) {
        AVLNode *n = (AVLNode *)calloc(1, sizeof(AVLNode));
        if (!n) return NULL;
        n->event = e; n->height = 1;
        return n;
    }
    if (event_cmp(&e, &root->event) < 0) root->left = avl_insert(root->left, e);
    else root->right = avl_insert(root->right, e);
    avl_update(root);
    int balance = avl_height(root->left) - avl_height(root->right);
    if (balance > 1 && event_cmp(&e, &root->left->event) < 0) return rotate_right(root);
    if (balance < -1 && event_cmp(&e, &root->right->event) > 0) return rotate_left(root);
    if (balance > 1 && event_cmp(&e, &root->left->event) > 0) {
        root->left = rotate_left(root->left); return rotate_right(root);
    }
}

```

```

    if (balance < -1 && event_cmp(&e, &root->right->event) < 0) {
        root->right = rotate_right(root->right); return rotate_left(root);
    }
    return root;
}

static void avl_reverse_inorder(const AVLNode *root, int *printed, int limit) {
    if (!root || *printed >= limit) return;
    avl_reverse_inorder(root->right, printed, limit);
    if (*printed < limit) {
        printf(" risk=%d ip=%s type=%s count-seq=%lu\n", root->event.risk, root->event.ip,
            root->event.type, root->event.seq);
        (*printed)++;
    }
    avl_reverse_inorder(root->left, printed, limit);
}

static void avl_free(AVLNode *root) {
    if (!root) return;
    avl_free(root->left); avl_free(root->right); free(root);
}

/* ----- Max priority queue ----- */
typedef struct {
    Event *a;
    size_t n, cap;
} MaxHeap;

static int higher(const Event *a, const Event *b) { return event_cmp(a, b) > 0; }
static void heap_swap(Event *a, Event *b) { Event t = *a; *a = *b; *b = t; }
static MaxHeap *heap_create(size_t cap) {
    MaxHeap *h = (MaxHeap *)calloc(1, sizeof(MaxHeap));
    if (!h) return NULL;
    h->cap = cap ? cap : 16;
    h->a = (Event *)malloc(h->cap * sizeof(Event));
    if (!h->a) { free(h); return NULL; }
    return h;
}

static int heap_push(MaxHeap *h, Event e) {
    if (h->n == h->cap) {
        size_t newcap = h->cap * 2;
        Event *tmp = (Event *)realloc(h->a, newcap * sizeof(Event));
        if (!tmp) return 0;
        h->a = tmp; h->cap = newcap;
    }
    size_t i = h->n++;
    h->a[i] = e;
    while (i > 0) {
        size_t p = (i - 1) / 2;
        if (!higher(&h->a[i], &h->a[p])) break;

```

```

        heap_swap(&h->a[i], &h->a[p]); i = p;
    }
    return 1;
}

static int heap_pop(MaxHeap *h, Event *out) {
    if (!h->n) return 0;
    *out = h->a[0];
    h->a[0] = h->a[--h->n];
    size_t i = 0;
    while (1) {
        size_t l = 2*i + 1, r = l + 1, best = i;
        if (l < h->n && higher(&h->a[l], &h->a[best])) best = l;
        if (r < h->n && higher(&h->a[r], &h->a[best])) best = r;
        if (best == i) break;
        heap_swap(&h->a[i], &h->a[best]); i = best;
    }
    return 1;
}

static void heap_free(MaxHeap *h) { if (h) { free(h->a); free(h); } }
/* ----- Engine ----- */
typedef struct {
    HashTable *hash;
    AVLNode *avl;
    MaxHeap *heap;
    unsigned long next_seq;
} Engine;

static Engine *engine_create(size_t hash_buckets) {
    Engine *e = (Engine *)calloc(1, sizeof(Engine));
    if (!e) return NULL;
    e->hash = hash_create(hash_buckets);
    e->heap = heap_create(16);
    if (!e->hash || !e->heap) { hash_free(e->hash); heap_free(e->heap); free(e); return NULL; }
    return e;
}

static int engine_process(Engine *e, Event ev) {
    ev.seq = e->next_seq++;
    hash_touch(e->hash, ev.ip);
    e->avl = avl_insert(e->avl, ev);
    return heap_push(e->heap, ev);
}

static void engine_free(Engine *e) {
    if (!e) return;
    hash_free(e->hash); avl_free(e->avl); heap_free(e->heap); free(e);
}
/* ----- Benchmark ----- */
static void make_ip(char *buf, unsigned long i, int skewed) {

```

```

    if (skewed) {
        /* 20% of IPs generate 80% of traffic. */
        unsigned long id = (i % 10 < 8) ? (i % 2000) : (2000 + i % 8000);
        snprintf(buf, MAX_IP_LEN, "10.%lu.%lu.%lu", (id >> 16) & 255UL, (id >> 8) &
255UL, id & 255UL);
    } else {
        unsigned long id = i % 900000;
        snprintf(buf, MAX_IP_LEN, "10.%lu.%lu.%lu", (id >> 16) & 255UL, (id >> 8) &
255UL, id & 255UL);
    }
}

static double elapsed(clock_t a, clock_t b) { return (double)(b - a) /
(double)CLOCKS_PER_SEC; }

static void benchmark_hash(size_t events, double load_factor, int skewed) {
    size_t distinct_target = skewed ? 3200 : 900000;
    size_t desired_items = events < distinct_target ? events : distinct_target;
    size_t buckets = (size_t)((double)desired_items / load_factor);
    if (buckets < 17) buckets = 17;
    HashTable *t = hash_create(buckets);
    if (!t) { fprintf(stderr, "Allocation failure\n"); exit(EXIT_FAILURE); }
    char ip[MAX_IP_LEN];
    clock_t t0 = clock();
    for (size_t i = 0; i < events; ++i) {
        make_ip(ip, (unsigned long)i, skewed);
        hash_touch(t, ip);
    }
    clock_t t1 = clock();
    volatile unsigned long sink = 0;
    for (size_t i = 0; i < events; ++i) {
        make_ip(ip, (unsigned long)(events - 1 - i), skewed);
        sink ^= hash_count(t, ip);
    }
    (void)sink;
    clock_t t2 = clock();
    printf("%zu,%f,%s,%zu,%zu,%f,%f,%zu\n",
        events, load_factor, skewed ? "skewed" : "uniform", t->size, t->items,
        elapsed(t0,t1), elapsed(t1,t2), t->collisions);
    hash_free(t);
}

static void run_benchmarks(void) {
    const size_t sizes[] = {10000, 100000, 1000000};
    const double loads[] = {0.50, 0.75, 0.90};
    puts("events,load_factor,distribution,buckets,distinct_ips,insert_seconds,lookup_seconds,collisio
n_events");
    for (size_t s = 0; s < sizeof(sizes)/sizeof(sizes[0]); ++s)
        for (size_t l = 0; l < sizeof(loads)/sizeof(loads[0]); ++l) {

```

```

        benchmark_hash(sizes[s], loads[l], 0);
        benchmark_hash(sizes[s], loads[l], 1);
    }
}

static void run_demo(void) {
    Engine *e = engine_create(101);
    if (!e) { fprintf(stderr, "Engine allocation failure\n"); exit(EXIT_FAILURE); }
    const Event demo[DEMO_EVENTS] = {
        {"192.168.1.10", 1001, "LOGIN_FAIL", 75, 0},
        {"10.0.0.5", 1002, "PORT_SCAN", 92, 0},
        {"192.168.1.10", 1003, "LOGIN_FAIL", 88, 0},
        {"10.0.0.7", 1004, "ACCESS_REQ", 45, 0},
        {"10.0.0.5", 1005, "BRUTE_FORCE", 98, 0},
        {"172.16.1.4", 1006, "DNS_ANOMALY", 64, 0},
        {"192.168.1.10", 1007, "LOGIN_FAIL", 90, 0},
        {"10.0.0.8", 1008, "ACCESS_REQ", 35, 0},
        {"10.0.0.5", 1009, "PORT_SCAN", 95, 0},
        {"172.16.1.4", 1010, "DNS_ANOMALY", 70, 0},
        {"10.0.0.7", 1011, "ACCESS_REQ", 52, 0},
        {"192.168.1.99", 1012, "MALWARE", 99, 0}
    };

    printf("\n=== Real-Time Cyber Threat Detection Engine ===\n");
    for (int i = 0; i < DEMO_EVENTS; ++i) engine_process(e, demo[i]);
    printf("Processed events      : %d\n", DEMO_EVENTS);
    printf("Distinct source IPs   : %zu\n", e->hash->items);
    printf("Hash collision events : %zu\n", e->hash->collisions);
    printf("Duplicate IP checks   : \n");
    const char *check_ips[] = {"192.168.1.10", "10.0.0.5", "8.8.8.8"};
    for (int i = 0; i < 3; ++i)
        printf(" %s -> %lu occurrence(s)\n", check_ips[i], hash_count(e->hash, check_ips[i]));
    printf("\nTop events from priority queue:\n");
    for (int i = 0; i < 5; ++i) {
        Event top;
        if (!heap_pop(e->heap, &top)) break;
        printf(" risk=%d ip=%s type=%s timestamp=%lu\n", top.risk, top.ip, top.type,
top.timestamp);
    }
    printf("\nHighest-risk ordered events from AVL tree:\n");
    int printed = 0;
    avl_reverse_inorder(e->avl, &printed, 5);
    engine_free(e);
}

int main(int argc, char **argv) {
    if (argc > 1 && strcmp(argv[1], "bench") == 0) run_benchmarks();
    else run_demo();
    return 0;
}

```

Actual execution output

```
=== Real-Time Cyber Threat Detection Engine ===
Processed events      : 12
Distinct source IPs  : 6
Hash collision events : 6
Duplicate IP checks  :
192.168.1.10 -> 3 occurrence(s)
10.0.0.5 -> 3 occurrence(s)
8.8.8.8 -> 0 occurrence(s)

Top events from priority queue:
risk=99 ip=192.168.1.99 type=MALWARE timestamp=1012
risk=98 ip=10.0.0.5 type=BRUTE_FORCE timestamp=1005
risk=95 ip=10.0.0.5 type=PORT_SCAN timestamp=1009
risk=92 ip=10.0.0.5 type=PORT_SCAN timestamp=1002
risk=90 ip=192.168.1.10 type=LOGIN_FAIL timestamp=1007

Highest-risk ordered events from AVL tree:
risk=99 ip=192.168.1.99 type=MALWARE count-seq=11
risk=98 ip=10.0.0.5 type=BRUTE_FORCE count-seq=4
risk=95 ip=10.0.0.5 type=PORT_SCAN count-seq=8
risk=92 ip=10.0.0.5 type=PORT_SCAN count-seq=1
risk=90 ip=192.168.1.10 type=LOGIN_FAIL count-seq=6

...Program finished with exit code 0
Press ENTER to exit console.
```

Figure 2 Actual execution output of the implemented C cyber threat detection engine.

The screenshot shows 12 processed events, 6 distinct source IPs, 6 reported hash-collision events, duplicate counts for repeated IPs, and the five highest-risk events returned by the priority queue and AVL tree. The program terminated with exit code 0.

Benchmark configurations:

Dataset Size	Load Factors	Distribution	Metrics
10,000 events	0.50, 0.75, 0.90	Uniform + skewed	Insertion time, lookup time, collisions
100,000 events	0.50, 0.75, 0.90	Uniform + skewed	Insertion time, lookup time, collisions
1,000,000 events	0.50, 0.75, 0.90	Uniform + skewed	Insertion time, lookup time, collisions

The benchmark produces machine-dependent timing values. The group should run benchmark mode on the actual submission environment and insert those measured values into the final results table rather than using timings from another machine.

6. Analysis and Engineering Decision

Load factor: Increasing the load factor reduces the number of buckets for a fixed number of distinct keys. This increases average chain length and can increase collision-related work. Lower load factors generally improve lookup behavior at the cost of additional memory.

Distribution: Uniform keys tend to spread across buckets more evenly. Skewed workloads repeatedly target a smaller set of IPs, increasing duplicate frequency and potentially changing collision behavior. The benchmark separates these two workload characteristics.

Scalability: Hashing is used for high-volume IP lookup because its expected average cost is $O(1)$. The AVL tree handles ordered risk reporting, while the heap handles immediate highest-risk retrieval.

Choice	Advantage	Limitation
Separate chaining	Simple collision handling; table remains usable as data grows	Longer chains and pointer overhead at high load

FNV-1a	Small and fast general-purpose string hash	Not a cryptographic hash
AVL tree	Predictable $O(\log n)$ ordered operations	Rotation and pointer overhead
Max heap	$O(1)$ highest-risk access after insertion	Not intended for arbitrary ordered traversal
Combined design	Each structure is matched to a specific operation	Multiple structures must remain consistent

Limitations and improvements:

- FNV-1a should not be treated as a cryptographic security primitive.
- The benchmark uses synthetic distributions rather than a production SOC log corpus.
- The current implementation is a single-process prototype without concurrent ingestion.
- Future work can compare open addressing with separate chaining, add streaming input, persistent storage, concurrency, alert thresholds, and stronger security controls.

7. Broader Considerations

The solution is relevant to SDG 9 through scalable digital infrastructure and performance engineering, and to SDG 16 through security monitoring and protection of digital systems.

A production deployment should apply access control, minimize unnecessary retention of network identifiers, protect logs against unauthorized access, and establish clear ethical and privacy policies. Performance claims should always be based on actual measurements from the stated environment.

8. Conclusion

A real-time cyber threat detection engine was implemented in C using a hash table with separate chaining, an AVL tree, and a max-priority queue. The actual execution demonstrates duplicate-IP detection, collision tracking, highest-risk event retrieval, and risk-ordered AVL reporting. The benchmark component supports controlled evaluation under different load factors, dataset sizes, and IP distributions.

The main remaining limitation is machine-dependent benchmark timing. Before submission, the team should execute the benchmark on its actual environment and record those results. Future improvements include open-addressing comparison, real log ingestion, concurrency, persistent storage, and production-grade security controls.

9. Student Reflection

This assignment required multiple data structures to cooperate in one application rather than treating each structure independently. The main learning is that data-structure selection should follow the dominant operation: hashing for membership and duplicate checks, AVL trees for ordered data, and heaps for repeated maximum-priority retrieval.

With additional resources, the project could be improved by implementing both collision-resolution approaches, automating repeated benchmark runs, plotting performance curves, adding a real log-stream input layer, and testing against a larger anonymized security-event dataset.

10. References

- K. Scarfone and P. Mell, *Guide to Intrusion Detection and Prevention Systems (IDPS)*, NIST Special Publication 800-94, National Institute of Standards and Technology, 2007.
- K. Kent and M. Souppaya, *Guide to Computer Security Log Management*, NIST Special Publication 800-92, National Institute of Standards and Technology, 2006.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- W. Stallings and L. Brown, *Computer Security: Principles and Practice*, 4th ed., Pearson, 2021.
- J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2022.
- D. Eastlake, T. Hansen, G. Fowler, K.-P. Vo, and L. Noll, *The FNV Non-Cryptographic Hash Algorithm*, RFC 9923, Internet Engineering Task Force, 2026.
- GNU Project, *GNU C Language Manual*, Free Software Foundation.
- GNU Project, *GCC Online Documentation*, GNU Compiler Collection.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, sections on hash tables, heaps, binary search trees, and algorithmic running-time analysis, MIT Press.
- L. Noll, G. Fowler, P. Vo, and related contributors, *Fowler–Noll–Vo (FNV) Hash Function*, FNV technical reference/source materials.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level / Marks
Problem formulation	CO2	PO1, PO2	L3/L4 10
Application of knowledge	CO2,CO3	PO1, PO2	L3/L4 20
Solution / Design / Implementation	CO3,CO4	PO3, PO5	L4/L5/L6 20

Modern tool usage	CO4,CO5	PO5	L3/L4 10
Validation	CO5	PO2, PO4	L4/L5 15
Analysis & justification	CO3,CO5	PO2, PO3	L4/L5 15
Broader considerations	CO5	PO6, PO7	L4/L5 5
Documentation & reflection	CO5	PO10	L3/L4 5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is not required to map to every PO.

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:
